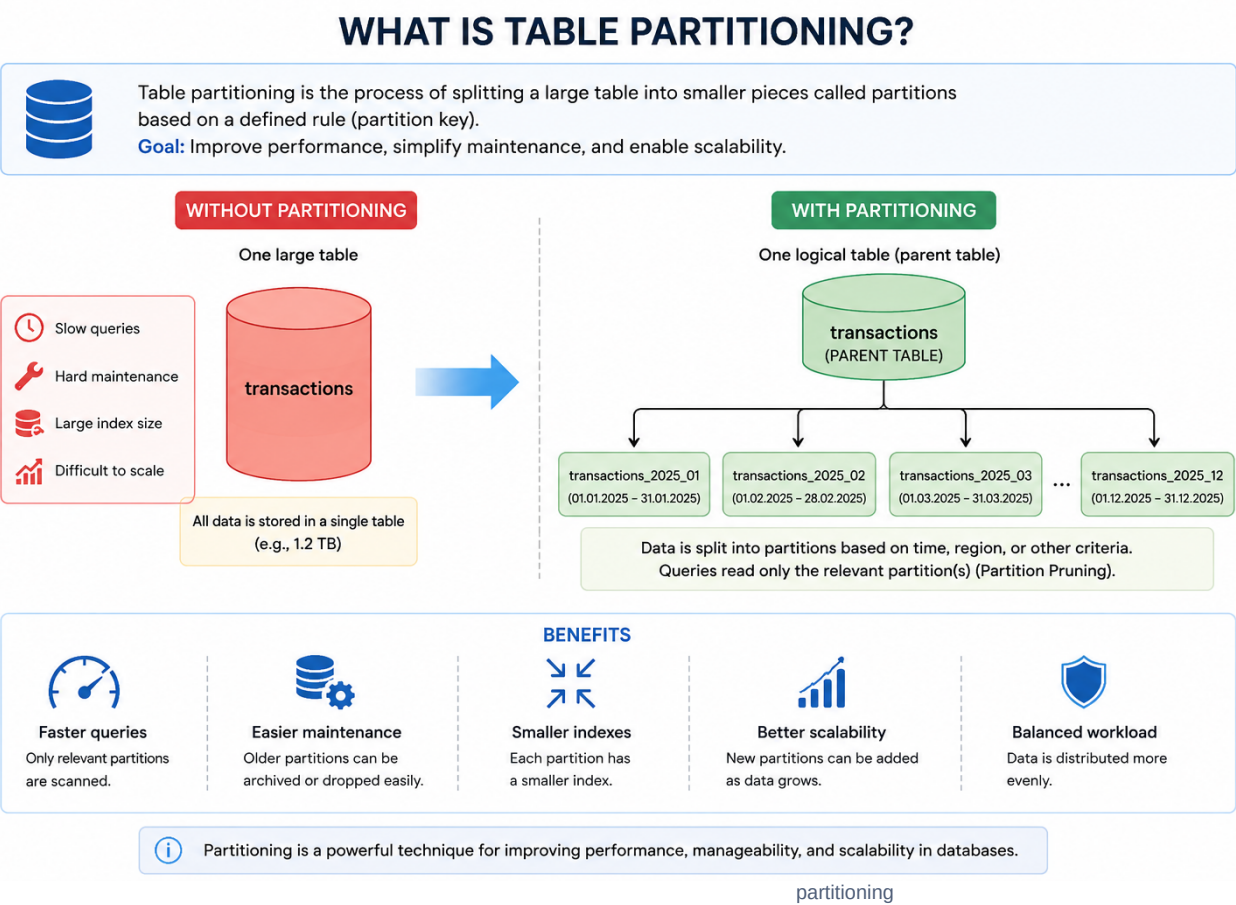


8. Partitioning

Objective

- How do we estimate data volume when designing a system?
- How much data can a regular PostgreSQL table store?
- How much read and write load can a regular table handle?
- Should we automatically use partitioning just because a table becomes large?
- Can an index solve the existing problem?
- How should a partition key be selected?
- Does partitioning really improve performance?
- What additional problems does partitioning create in production?

Partitioning is the process of splitting one large logical table into multiple smaller physical tables (partitions) based on a partition key. Applications continue querying a single parent table, while PostgreSQL automatically routes reads and writes to the appropriate partition.



partitioning

1. Business Requirement

Database design should start from the business process, not from the table.

Suppose we are building a system that stores customer transactions in a bank.

The business requires the following capabilities:

- customers should be able to view their latest transactions
- the finance team should be able to generate monthly reports
- the fraud team should be able to investigate transactions within a specific date range
- data should be retained for seven years
- old data should be archivable

- transaction history in the mobile application should open within two seconds

Possible table:

```
CREATE TABLE transactions (
  transaction_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id BIGINT NOT NULL,
  account_id BIGINT NOT NULL,
  amount NUMERIC(18, 2) NOT NULL,
  currency VARCHAR(3) NOT NULL,
  transaction_type VARCHAR(30) NOT NULL,
  status VARCHAR(30) NOT NULL,
  created_at TIMESTAMP NOT NULL
);
```

Main business query:

```
SELECT
  transaction_id,
  account_id,
  amount,
  currency,
  transaction_type,
  status,
  created_at
FROM transactions
WHERE customer_id = :customerId
  AND created_at >= :fromDate
  AND created_at < :toDate
ORDER BY created_at DESC
LIMIT 20;
```

At this stage, we do not make a partitioning decision yet.

First, we identify business behavior:

Business operation	Main filter
Customer transaction history	customer_id, created_at
Account statement	account_id, created_at
Fraud investigation	account, card, date
Monthly financial reporting	created_at
Retention and archive	created_at
Lookup by transaction reference	transaction_reference

An initial conclusion can be drawn from this:

Many critical processes are date-based.

However, this does not yet mean:

We should partition by created_at

First, we need to analyze the data volume and the capabilities of a regular PostgreSQL table.

The main flow of the topic is:

Business Requirement
Capacity Estimation
Current or Expected Problems
Benchmark
Production Concerns
Should We Partition?
Choose Partition Key
Choose Partition Strategy
PostgreSQL Implementation

2. Capacity Estimation

Capacity estimation attempts to predict future load before the system goes into production.

These estimates are not exact predictions of the future. They are an initial model used to support architectural decisions.

2.1 Calculating Write Volume

Suppose:

Average write rate: 100 transactions/second
Peak write rate: 500 transactions/second
Each transaction: 1 main row
Retention: 7 years

Average daily row count:

$100 \times 86,400$
 $= 8,640,000$ rows/day

Annual row count:

$8,640,000 \times 365$
3.15 billion rows/year

Seven-year retention:

$3.15 \text{ billion} \times 7$
22 billion rows

An important detail here is that we should not only look at average load.

Peak load should be calculated separately.

For example:

500 writes/second for 4 hours per day
60 writes/second for the remaining 20 hours

Calculation:

Peak period:

$500 \times 4 \times 3,600$
 $= 7,200,000$ rows

Normal period:

$$60 \times 20 \times 3,600$$

$$= 4,320,000 \text{ rows}$$

Total:

$$11,520,000 \text{ rows/day}$$

Therefore, the real daily growth may be different from the average model of 100 writes per second.

2.2 Calculating Read Volume

Write capacity tells us how much data will be stored in the database. However, estimating read capacity requires understanding **how frequently that data will be accessed**.

Suppose the system stores approximately:

- **11.5 million new transactions per day** (calculated in the previous section)

Assume that historical transaction data is accessed in the following way:

- Every newly created transaction is viewed **4 times on average**
 - Customer mobile application
 - Internet banking
 - Customer support
 - Internal reporting or reconciliation

Therefore:

Daily Reads

$$11,520,000 \times 4$$

$$= 46,080,000 \text{ reads/day}$$

Average reads per second:

$$46,080,000$$

$$86,400$$

$$533 \text{ reads/second}$$

However, production systems rarely experience uniform traffic throughout the day.

Suppose business analytics show the following pattern:

- Peak hours (09:00–13:00): **2,000 reads/second**
- Remaining hours: **250 reads/second**

Daily read volume becomes:

Peak period:

$$2,000 \times 4 \times 3,600$$

$$=$$

$$28,800,000 \text{ reads}$$

Normal period:

$$250 \times 20 \times 3,600$$

=

18,000,000 reads

Total:

46,800,000 reads/day

This aligns closely with the estimation derived from business activity.

Concurrent Database Load

Now assume that the average execution time of a read query is **15 ms**.

Using Little's Law:

Concurrent Queries

Requests/sec × Query Duration

=

$$2,000 \times 0.015$$

=

30 concurrent queries

If the same query becomes slower because of missing indexes or larger tables:

Average Query Time

=

500 ms

Then:

$$2,000 \times 0.5$$

=

1,000 concurrent queries

Although the business traffic has **not changed**, the database now has to maintain over **30 times** more active queries simultaneously.

This demonstrates why query latency directly impacts database capacity.

Slow Query

Connections remain occupied longer

Concurrent queries increase

CPU utilization rises

More buffer cache misses occur

Disk I/O increases

Overall database throughput decreases

2.3 Calculating Table Size

Simplified formula:

Table size

row count × average physical row size

However, in PostgreSQL, row size is not only the sum of business columns. Physically, each row and its underlying storage structures contain additional metadata and overhead.

- **Tuple Header** — Internal metadata stored for each row. It contains xmin, xmax, and other management data required for MVCC.
- **Page Metadata** — Each 8 KB data page stores its own header information. This metadata helps PostgreSQL manage free space and tuple locations within the page.
- **Dead Tuples** — After UPDATE and DELETE operations, old row versions are not immediately removed. They remain temporarily on disk and are later cleaned by VACUUM.
- **Index Entries** — Each index stores a separate entry for the row. The more indexes a table has, the higher the storage and write cost.
- **TOAST Data** — Large values such as TEXT, JSONB, and BYTEA are stored outside the main table in special TOAST storage, while only a pointer to the data is stored inside the row.
- **Bloat** — Over time, UPDATE and DELETE operations create unused areas that still occupy disk space. This unused additional volume is called bloat.

Therefore, logical row size and physical row size are not the same.

For example, suppose the estimated physical size of a business row is 180 bytes:

3.15 billion rows × 180 bytes
567 GB/year

For seven years:

567 GB × 7
3.97 TB heap data

This is only the table heap.

We should also consider indexes.

For example:

```
CREATE INDEX idx_transactions_customer_created
ON transactions(customer_id, created_at DESC);
```

```
CREATE INDEX idx_transactions_account_created
ON transactions(account_id, created_at DESC);
```

```
CREATE INDEX idx_transactions_reference
ON transactions(transaction_reference);
```

3. How Much Data Can a Regular PostgreSQL Table Handle?

The correct answer to this question is:

There is no universal limit such as:

A table becomes large after 10 million rows

for PostgreSQL.

PostgreSQL technical hard limits and practical performance limits are different. PostgreSQL documentation also notes that practical limits such as disk capacity, workload, and performance may be reached before theoretical limits.

A regular, non-partitioned PostgreSQL table can successfully manage hundreds of millions or even billions of rows depending on:

- row size
- query patterns
- index design
- RAM
- storage type
- CPU
- write concurrency
- number of connections
- replication

Therefore, the following approach is incorrect:

The table reached 100 million rows

Partitioning is mandatory

The correct approach is:

The table reached 100 million rows

What is the latency of critical queries?

Does the index working set fit into RAM?

Can autovacuum keep up?

Is retention expensive?

Is the maintenance SLA being violated?

Will partitioning solve these problems?

1. **What is the latency of critical queries?** — If the most important queries still operate within the SLA, partitioning may not be required only because of the row count.
2. **Does the index working set fit into RAM?** — If the actively used portion of the index does not fit into RAM, disk reads increase and performance may decrease.
3. **Can autovacuum keep up?** — If autovacuum cannot clean dead tuples in time, bloat appears and queries become slower.
4. **Is retention expensive?** — If deleting old data takes hours, partitioning may simplify this process significantly through `DROP PARTITION`.
5. **Is the maintenance SLA being violated?** — If `VACUUM`, `REINDEX`, and other maintenance operations do not fit into the allocated maintenance window, partitioning may help divide these operations into smaller units.
6. **Will partitioning solve these problems?** — This is the final decision. If the problem is related to query patterns, retention, or maintenance, partitioning may be useful. If the problem is poor SQL, an incorrect index, or insufficient resources, partitioning will not help.

4. Current or Expected Problems

Capacity estimation and normal-table analysis are used to identify concrete problems.

Problem 1: Critical Query Latency Increases

For example:

Customer history p95:
2025: 80 ms
2026: 190 ms
2027 projection: 600 ms
SLA: 300 ms

The following should be determined:

- whether the index is selected correctly
- whether the query uses a date range
- whether the index is being evicted from cache
- whether the table has bloat
- whether the data distribution has changed

Problem 2: Retention Delete Is Expensive

Every month, 250 million old rows must be deleted.

Regular table:

```
DELETE FROM transactions
WHERE created_at >= :monthStart
AND created_at < :nextMonth;
```

Partitioned design instead:

```
ALTER TABLE transactions
DETACH PARTITION transactions_2019_01;
```

This is a very strong partitioning use case.

Problem 3: Maintenance Window Is Violated

Global index rebuild: 9 hours
Maintenance window: 2 hours
Partition index rebuild:
One monthly index: 20 minutes

Here, partitioning provides an operational benefit.

Problem 4: Current and Historical Data Behave Differently

Current month:
high read/write

Previous year:
mostly read

Old years:
archive-only

A regular table stores all lifecycle stages inside one physical object.

Partitioning allows them to be managed separately.

5. Should We Partition?

At this stage, the partitioning decision is made.

5.1 Strong Signals for Partitioning

Partitioning is a strong candidate if:

- the table is growing rapidly
- critical queries use the partition key
- retention is period-based
- historical and current data have different lifecycles
- full-table maintenance violates the SLA
- indexes are operationally too large
- ownership is associated with a stable region, tenant, or legal entity
- the projected number of partitions is manageable

5.2 Reasons Not to Partition

Partitioning may not be necessary if:

- the table is still small and stable
- the problem is a missing index
- queries do not use the partition key
- most queries scan all data
- the partition key changes frequently
- one partition will contain 95% of the data
- there is no retention requirement
- maintenance works comfortably on the regular table
- benchmarks do not show benefits for the partitioned model

6. Choose Partition Key

Partition-key selection is the most important part of the entire design.

6.1 Date Key

It is appropriate if:

- queries use date ranges
- retention is date-based
- data is append-heavy
- historical data changes less frequently
- reporting is period-based

Possible keys:

```
created_at
business_date
transaction_date
event_time
```

The domain difference is important here.

For example, in banking:

```
created_at:
when the row was written to the database
```

```
business_date:
which banking day the transaction belongs to
```

If reporting and retention operate based on `business_date`, then `created_at` may be the wrong partition key.

6.2 Region Key

It is appropriate if:

- queries filter by region
- ownership is region-based
- authorization is region-scoped
- the region value is stable
- regional operations are a critical workload

However:

```
current_owner_region
```

is a risky key if it changes frequently.

A more stable key may be:

```
registration_region
```

7. Choose Partition Strategy

7.1 Range Partitioning

`PARTITION BY RANGE (created_at)`

Appropriate for:

- transactions
- audit events
- application logs
- payment attempts
- invoices
- statement data

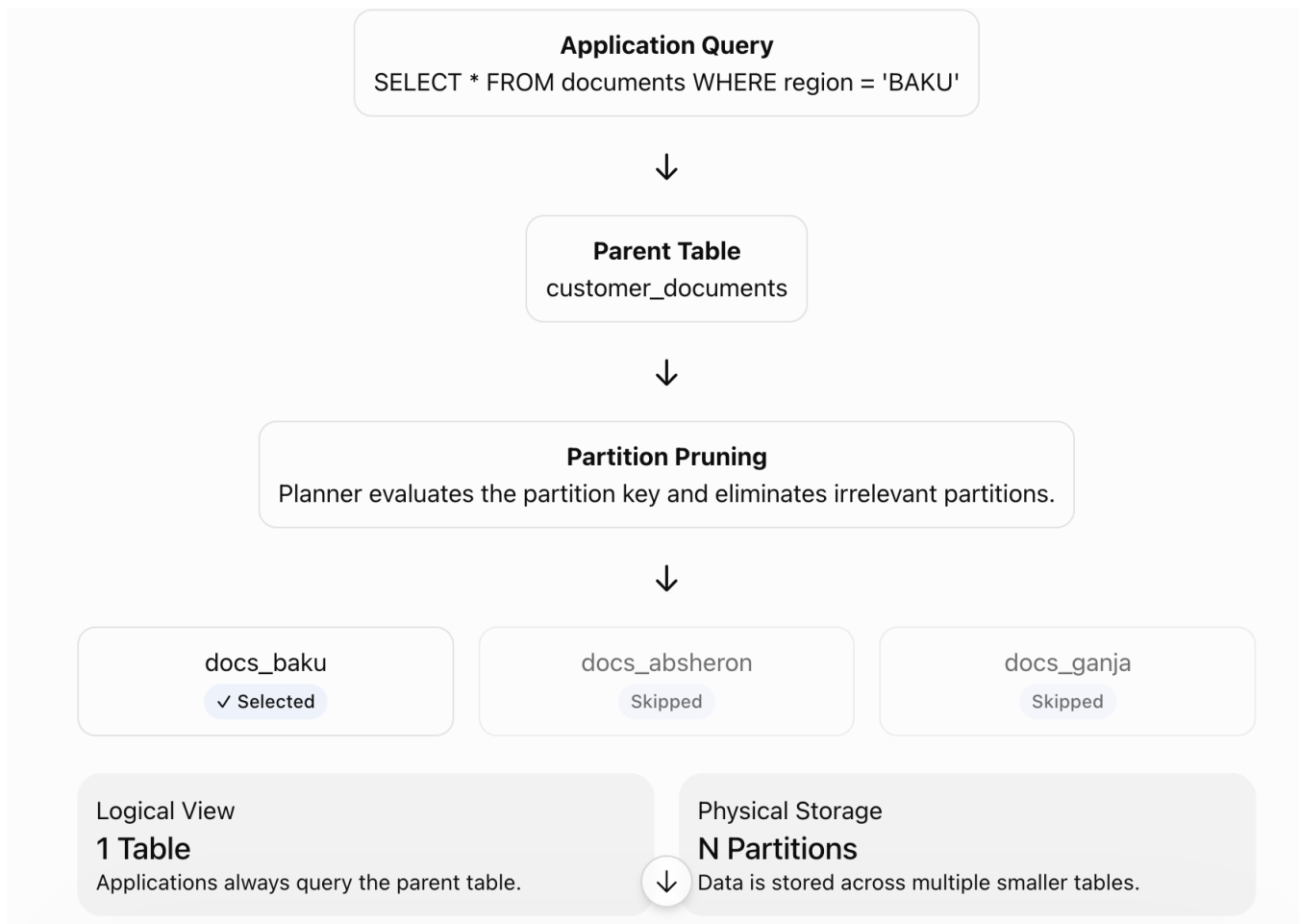
7.2 List Partitioning

`PARTITION BY LIST (region_code)`

Appropriate for:

- regional documents
- legal entities
- stable business units
- source systems
- a small number of large tenants

8. PostgreSQL Implementation



8.1 Regular Table

```
CREATE TABLE transactions_regular (  
    transaction_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id BIGINT NOT NULL,  
    account_id BIGINT NOT NULL,  
    amount NUMERIC(18, 2) NOT NULL,  
    currency VARCHAR(3) NOT NULL,  
    transaction_type VARCHAR(30) NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    created_at TIMESTAMP NOT NULL  
);
```

Index:

```
CREATE INDEX idx_regular_customer_created  
ON transactions_regular(customer_id, created_at DESC);
```

8.2 Partitioned Table

```
CREATE TABLE transactions_partitioned (  
    transaction_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    customer_id BIGINT NOT NULL,  
    account_id BIGINT NOT NULL,  
    amount NUMERIC(18, 2) NOT NULL,  
    currency VARCHAR(3) NOT NULL,  
    transaction_type VARCHAR(30) NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    created_at TIMESTAMP NOT NULL,  
    PRIMARY KEY (transaction_id, created_at)  
) PARTITION BY RANGE (created_at);
```

In a partitioned table, a parent-level unique or primary-key constraint generally has to include all partition-key columns.

8.3 Monthly Partitions

```
CREATE TABLE transactions_2026_06  
PARTITION OF transactions_partitioned  
FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
```

```
CREATE TABLE transactions_2026_07  
PARTITION OF transactions_partitioned  
FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
```

```
CREATE TABLE transactions_2026_08  
PARTITION OF transactions_partitioned  
FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
```

8.4 Insert Routing

```
INSERT INTO transactions_partitioned (  
    customer_id,  
    account_id,  
    amount,  
    currency,  
    transaction_type,  
    status,  
    created_at  
)  
VALUES (  
    101,  
    5001,  
    250.00,  
    'AZN',  
    'TRANSFER',  
    'SUCCESS',  
    TIMESTAMP '2026-07-10 12:00:00'  
);
```

Logical flow:

Read created_at
Compare partition bounds
2026-07-10 matches July
Insert into transactions_2026_07

8.5 Missing Partition

If a partition for 2027 has not been created:

```
INSERT INTO transactions_partitioned (  
    customer_id,  
    account_id,  
    amount,  
    currency,  
    transaction_type,  
    status,  
    created_at  
)  
VALUES (  
    101,  
    5001,  
    100,  
    'AZN',  
    'TRANSFER',  
    'SUCCESS',  
    TIMESTAMP '2027-01-10'  
);
```

Result:

ERROR: no partition of relation found for row

This may cause a write outage in production.

8.6 Query Without Partition Key

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT *  
FROM transactions_partitioned  
WHERE customer_id = 101;
```

created_at is not present.

PostgreSQL does not know in which months the customer's data exists.

The result may therefore be:

```
Append  
-> January index lookup  
-> February index lookup  
-> March index lookup  
-> ...
```

Even if every partition has an index, all partitions may still be visited.

9. Benchmark

Partitioning should be selected through measurement, not through performance assumptions.

9.1 Test Data

```

INSERT INTO transactions_regular (
    customer_id,
    account_id,
    amount,
    currency,
    transaction_type,
    status,
    created_at
)
SELECT
    1 + floor(random() * 100000)::BIGINT,
    1 + floor(random() * 200000)::BIGINT,
    round((1 + random() * 10000)::NUMERIC, 2),
    (ARRAY['AZN', 'USD', 'EUR'])[1 + floor(random() * 3)::INT],
    (ARRAY[
        'TRANSFER',
        'CARD_PAYMENT',
        'CASH_OUT'
    ])[1 + floor(random() * 3)::INT],
    (ARRAY[
        'SUCCESS',
        'FAILED',
        'PENDING'
    ])[1 + floor(random() * 3)::INT],
    TIMESTAMP '2026-01-01'
    + random() * (TIMESTAMP '2027-01-01' - TIMESTAMP '2026-01-01')
FROM generate_series(1, 5000000);

```

The same data is copied into the partitioned table:

```

INSERT INTO transactions_partitioned (
    customer_id,
    account_id,
    amount,
    currency,
    transaction_type,
    status,
    created_at
)
SELECT
    customer_id,
    account_id,
    amount,
    currency,
    transaction_type,
    status,
    created_at
FROM transactions_regular;

```

10. Production Concerns

10.1 Future Partition Provisioning

Partitions should be created in advance.

For example:

Current date: July 2026
 Existing partitions:
 July, August, September

A scheduled job or migration process should create future partitions.

Provisioning should include:

- partition table
- index
- constraint
- grants
- ownership
- tablespace
- monitoring
- statistics

10.2 Hot Partition

With time-based partitioning, all current writes go to the current-month partition.

Historical partitions:

read-only

Current partition:

all inserts
all updates
most recent reads

Partitioning does not automatically distribute load across different servers.

All partitions may exist on the same PostgreSQL instance.

Hot-partition problems include:

- index contention
- high WAL volume
- autovacuum pressure
- write latency
- storage saturation

Possible solutions:

- weekly or daily partitions
- sub-partitioning
- removing unnecessary indexes
- batch writes
- autovacuum tuning
- faster storage
- workload redesign

10.3 Updating the Partition Key

```
UPDATE office_documents
SET region_code = 'ABSHERON'
WHERE document_id = 1001;
```

If region_code is the partition key, PostgreSQL must move the row from one physical partition to another.

Although this is logically an update, physically it creates delete-and-insert-like behavior.

The partition key should be stable.

11. Final Decision Model

The partitioning decision should be made using the following comparison:

Partitioning benefits:

Partition pruning
+ lifecycle isolation
+ fast retention
+ smaller maintenance units
+ active-data locality
+ ownership alignment

versus

Partitioning costs:

planning overhead
+ schema complexity
+ automation
+ missing-partition risk
+ cross-partition query cost
+ more physical objects
+ operational burden

Final flow:

Identify the business workflow
Calculate read and write workload
Estimate future row count and storage volume
First build a regular table with proper indexes
Benchmark with realistic data
Measure query, vacuum, retention, and maintenance problems
Check whether partitioning solves the problem
Calculate partition-key coverage
Prepare partition-size and partition-count projections
Compare regular and partitioned designs
Choose the simplest design that meets the SLA

The main conclusion:

The number of rows that a PostgreSQL table stores is not the main criterion for making a partitioning decision.

The main criteria are:

- how data is read
- how data is written
- how large the active working set is
- how large indexes become
- whether autovacuum can keep up with the workload
- how retention is performed
- whether maintenance completes within the SLA
- whether queries use the partition key
- whether benchmarks show a measurable advantage

Partitioning is not the first optimization.

The following should be checked first:

Correct SQL

Correct indexes

Correct statistics

Vacuum health

Memory and storage

Connection and transaction design

Read replica or reporting architecture

Partitioning

Partitioning should only be selected when it solves a concrete business or operational problem and simpler solutions are not sufficient.